Task 01 – Network Scanning and Automation using Nmap and Python

CyArt SOC Internship – Week 01

Intern Name: Harvinder Singh
Date: 16 March 2026

1. Introduction

This report explains the work I completed for Task 01 during my SOC internship at CyArt. The goal of this task was to understand basic networking concepts, use Nmap to scan a target machine, analyze the results, and then automate the scan using a Python script.

To do this, I built a small virtual lab where Kali Linux acted as the attacker or scanner machine and Metasploitable 2 acted as an intentionally vulnerable target. I first ran different Nmap scans (SYN, TCP connect, and UDP) on the target and reviewed the results to see which ports and services were exposed. After that, I wrote a Python script using the python-nmap library to run a SYN scan automatically and save the findings into a text report. This helped me connect theory (IP addresses, ports, protocols) with real hands-on practice in a safe environment.

2. Lab Environment and Target Details

For this task I used the following setup:

- Host Machine: Windows with Oracle VirtualBox installed
- Attacker VM: Kali Linux
- Target VM: Metasploitable 2
- Virtual Network: NAT Network named “Hacking_Lab”
- Subnet: 192.168.1.0/24
- Kali IP Address: 192.168.1.4
- Metasploitable IP Address: 192.168.1.3

Kali and Metasploitable were both connected to the same NAT Network inside VirtualBox. This network, called “Hacking_Lab”, created an isolated lab where I could safely run scans without touching any real production systems. Kali was used to launch the scans, and Metasploitable 2 was used as a deliberately vulnerable target with many open services.

3. Nmap Commands Used

In this task I ran three different Nmap scans against the target IP address 192.168.1.3 and saved each result to a separate text file:

1. **SYN Scan (half-open TCP scan)**
   nmap -sS 192.168.1.3 -oN syn_scan.txt

   This scan sends TCP SYN packets and decides whether a port is open based on the response, without completing the full TCP handshake. It is relatively fast and a bit less noisy compared to a full TCP connect scan.

2. **TCP Connect Scan (full handshake)**
   nmap -sT 192.168.1.3 -oN tcp_scan.txt

   This scan completes the full three-way TCP handshake with each port. It gives similar results on this target, but it is easier for firewalls, IDS, and log systems to detect because full connections are made.

3. **UDP Scan**
   nmap -sU 192.168.1.3 -oN udp_scan.txt

   This scan looks for UDP services. UDP scanning is slower and its results are sometimes less clear, because many UDP services do not respond and some firewalls silently drop packets. As a result, some ports may appear as open | filtered instead of clearly open or closed.

These commands gave me a good view of which TCP and UDP services were exposed on the Metasploitable machine and formed the basis for the later analysis and automation.

4. Scan Results: Open Ports and Services

The Nmap SYN scan showed that the target host 192.168.1.3 was online and that 977 TCP ports were closed, while a number of important ports were open. Using both the manual SYN scan output (syn_scan.txt) and the automated Python report (scan_report.txt), I created the following summary of open TCP ports and their services:

Port	Protocol	Service	Notes / Potential Risk (High Level)
21	TCP	ftp	Uses cleartext credentials and is vulnerable to brute-force attacks.
22	TCP	ssh	Provides remote shell access; weak or reused passwords can lead to full compromise.
23	TCP	telnet	Remote access without encryption; credentials and commands can be sniffed.
25	TCP	smtp	Mail service; misconfiguration can allow spam, spoofing, or open relay.
53	TCP	domain	DNS service; bad configurations can leak internal information.
80	TCP	http	Web server; common entry point for XSS, SQL injection, and other web attacks.
111	TCP	rpcbind	Exposes RPC services that may reveal or open further attack surface.
139	TCP	netbios-ssn	Used for legacy Windows file sharing and enumeration.
445	TCP	microsoft-ds	SMB file sharing; known for serious vulnerabilities like EternalBlue-style exploits.
512	TCP	exec	Remote command execution service; dangerous if reachable by attackers.
513	TCP	login	Remote login service; can expose user authentication.
514	TCP	shell	Remote shell; high-risk if it is left open to untrusted networks.
1099	TCP	rmiregistry	Java RMI registry; can be abused for remote code execution in some setups.
1524	TCP	ingreslock	Often used as a backdoor port on Metasploitable systems.
2049	TCP	nfs	Network File System; misconfiguration can expose shared files.

Port	Protocol	Service	Notes / Potential Risk (High Level)
2121	TCP	ccproxy-ftp	Alternative FTP service with similar risks as standard FTP.
3306	TCP	mysql	Database service; weak credentials can expose or damage stored data.
5432	TCP	postgresql	Database service; needs strict access control to prevent data leaks.
5900	TCP	vnc	Remote desktop access; weak passwords give full graphical control.
6000	TCP	X11	X Window System; can allow session hijacking or screen capture.
6667	TCP	irc	IRC service; can be used as a command-and-control channel.
8009	TCP	ajp13	Apache JServ Protocol; can be abused in some Tomcat-related exploits.
8180	TCP	(unknown)	HTTP-like service that requires further enumeration.

These results confirm that Metasploitable 2 is intentionally configured with many exposed services, which makes it a good target for learning about scanning and basic exploitation.

5. Analysis of Scan Types

On this target, the SYN scan (-sS) and the TCP connect scan (-sT) reported the same set of open TCP ports. The difference is mainly in how they work. The SYN scan sends SYN packets and checks the responses without completing the TCP handshake, so it is fast and a bit less obvious in logs. The TCP connect scan completes the full three-way handshake for each port, which is more reliable in some environments but easier to detect by firewalls and intrusion detection systems.

The UDP scan (-sU) behaved differently. It was slower and some ports appeared as open | filtered, which means Nmap could not clearly tell if the ports were open or blocked by a firewall. This is a normal challenge with UDP scanning, because many UDP services do not send replies and some security devices silently drop the packets. Even with these

limitations, the UDP scan still helped highlight that the target might be running UDP-based services that would need deeper testing.

6. Service Research and Common Vulnerabilities

6.1 Telnet (Port 23/TCP)

Telnet provides remote command-line access but does not use any encryption. All usernames, passwords, and commands are sent in plain text across the network. If an attacker can watch the traffic, they can easily capture credentials and reuse them to log in. Because of this, Telnet is considered unsafe on modern networks and is normally replaced by SSH. Having Telnet open on a real production server would be a serious security issue.

6.2 HTTP (Port 80/TCP)

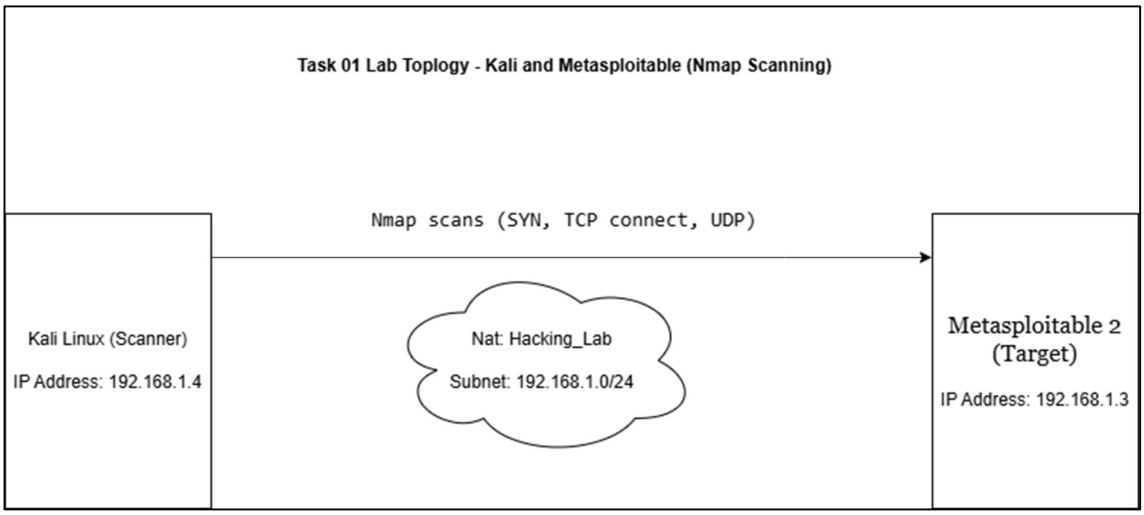
HTTP is used to serve web pages and web applications. The main security risks come from the application layer, not just the protocol itself. Common problems include SQL injection, cross-site scripting (XSS), insecure file uploads, and misconfigured access controls. Attackers often scan HTTP services first, fingerprint the server and frameworks, and then try known exploits or fuzzing tools to find vulnerabilities. A poorly protected web application can become an easy entry point into the whole system.

6.3 FTP (Port 21/TCP)

FTP is a file transfer protocol that also typically sends credentials and data in cleartext. This makes it easy for an attacker with network access to capture usernames and passwords or to perform brute-force login attempts. Some FTP servers also allow anonymous access or have weak permissions, which can leak sensitive files or allow unauthorized uploads. On a production network, FTP should either be replaced with secure alternatives like SFTP or be tightly restricted and monitored.

7. Network Diagram

To make the setup easy to understand, I created a simple network diagram for this lab. The diagram shows Kali Linux on the left as the scanner, Metasploitable 2 on the right as the target, and the VirtualBox NAT Network in the middle. The NAT Network is named **"Hacking_Lab"** and uses the subnet **192.168.1.0/24**. Kali has the IP address **192.168.1.4** and Metasploitable has **192.168.1.3**. An arrow from Kali to Metasploitable represents the Nmap scans (SYN, TCP connect, and UDP).



- *Figure 1: Lab topology – Kali Linux scanning the Metasploitable 2 target over the "Hacking_Lab" NAT Network.*

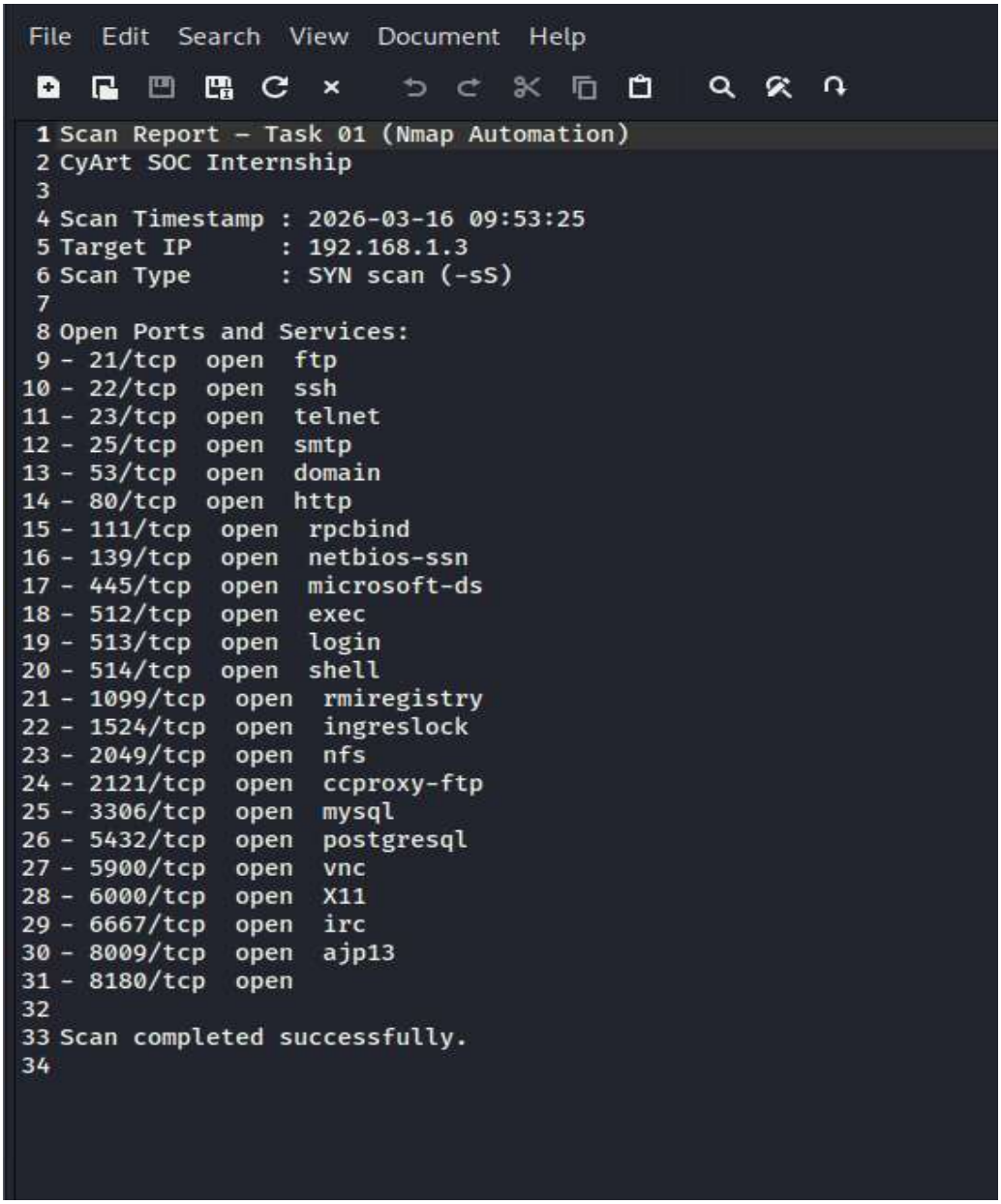
8. Python-Based Nmap Automation

After completing the manual Nmap scans, I automated part of the process using a Python script named nmap_automation.py. The script uses the python-nmap library to control Nmap from Python. When I run the script, it asks me to enter a target IP address (in this lab, 192.168.1.3). It then runs a SYN scan (-sS) against that target, parses the Nmap results, and collects all open TCP ports and their service names.

Once the scan is finished, the script writes the findings into a text file called scan_report.txt. The report includes the scan timestamp, the target IP, the scan type, and a list of open ports with their services, followed by a completion message. This turns the manual scanning workflow into a repeatable step that I can run any time with a single command. It is a small but practical example of how scripting can save time and reduce human error during basic security checks.

9. Sample Automation Output (scan_report.txt)

This section shows the output produced by my Python automation script. After I enter the target IP address, the script runs a SYN scan using Nmap and writes the results into a text file named scan_report.txt. The file includes the timestamp of the scan, the target IP, the scan type, and a list of all open TCP ports with their services. This helps me quickly review the results without re-running the Nmap command manually.



```
Scan Report – Task 01 (Nmap Automation)
CyArt SOC Internship

Scan Timestamp : 2026-03-16 09:53:25
Target IP      : 192.168.1.3
Scan Type      : SYN scan (-sS)

Open Ports and Services:
- 21/tcp  open  ftp
- 22/tcp  open  ssh
- 23/tcp  open  telnet
- 25/tcp  open  smtp
- 53/tcp  open  domain
- 80/tcp  open  http
- 111/tcp  open  rpcbind
- 139/tcp  open  netbios-ssn
- 445/tcp  open  microsoft-ds
- 512/tcp  open  exec
- 513/tcp  open  login
- 514/tcp  open  shell
- 1099/tcp  open  rmiregistry
- 1524/tcp  open  ingreslock
- 2049/tcp  open  nfs
- 2121/tcp  open  ccproxy-ftp
- 3306/tcp  open  mysql
- 5432/tcp  open  postgresql
- 5900/tcp  open  vnc
- 6000/tcp  open  X11
- 6667/tcp  open  irc
- 8009/tcp  open  ajp13
- 8180/tcp  open

Scan completed successfully.
```

- *Figure 2: Screenshot of the automatically generated scan_report.txt file showing the SYN scan results for 192.168.1.3 (open ports and services).*

10. Key Learnings

From this task I learned:

- How to build a small, isolated lab using VirtualBox, Kali Linux, and Metasploitable for safe security testing.
- How to use different Nmap scan types (SYN, TCP connect, and UDP) and understand the strengths and limitations of each.
- How to read Nmap output to identify open ports, running services, and basic risk areas on a target system.
- Why services like Telnet, FTP, SMB, and insecure web applications are dangerous when exposed without proper hardening.
- How to use Python and the python-nmap library to automate Nmap scans and generate a consistent text report for documentation.

This single task combined lab setup, manual scanning, analysis, script automation, and documentation, which are all core skills for a SOC analyst and for anyone starting in practical cybersecurity.